Test Case 1

University Scholarship Allocation System Using Python

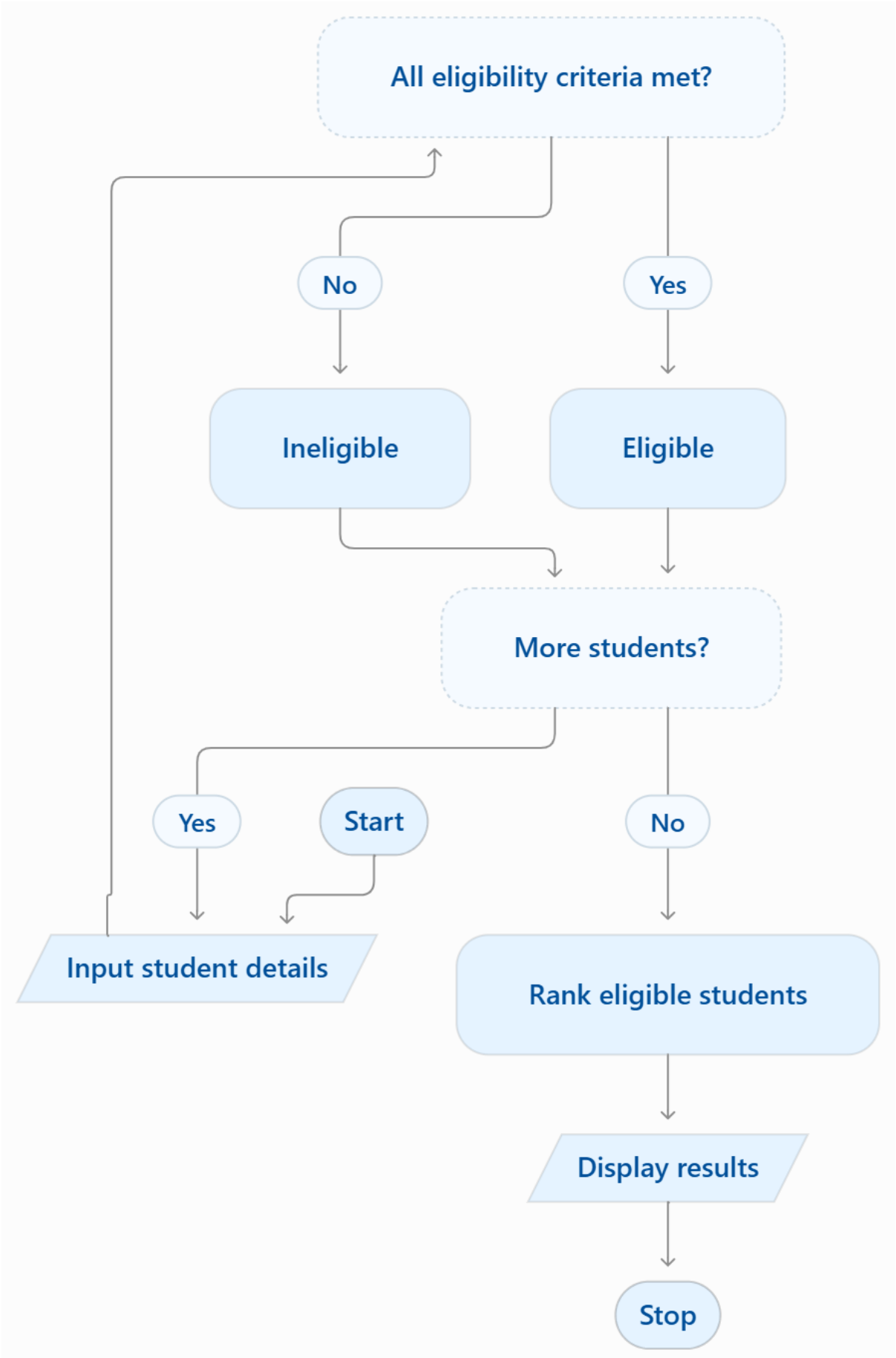
1. Problem Statement

University Scholarship Allocation: Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

2. Algorithm

1. Start the program.
2. Read the number of student applicants.
3. For each student, input the name, aggregate marks, subject-wise marks, attendance percentage, and annual family income.
4. Check whether the aggregate marks meet the minimum requirement.
5. Check whether every subject mark meets the minimum requirement.
6. Check whether attendance meets the minimum requirement.
7. Check whether annual family income is within the permitted limit.
8. If all conditions are satisfied, add the student to the eligible list. Otherwise, mark the student as ineligible.
9. Sort eligible students using the following priorities:
    - Aggregate marks in descending order.
    - Attendance in descending order.
    - Annual family income in ascending order.
    - Name in alphabetical order.
10. Display the ranked eligible students and their details.
11. Display the ineligible students and their reasons.
12. Stop the program.

3. Flowchart


All eligibility criteria met?
No
Yes
Ineligible
Eligible
More students?
Yes
Start
No
Input student details
Rank eligible students
Display results
Stop

4.source code

```
n = int(input("Number of students: "))
eligible = []
ineligible = []

for _ in range(n):
    name = input("Name: ")
    marks = float(input("Aggregate marks: "))
    subjects = list(map(float, input("Subject marks: ").split()))
    attendance = float(input("Attendance (%): "))
    income = float(input("Family income: "))

    if marks >= 75 and min(subjects) >= 60 and attendance >= 75 and income <= 250000:
        eligible.append((name, marks, attendance, income))
    else:
        ineligible.append(name)

eligible.sort(key=lambda s: (-s[1], -s[2], s[3], s[0].lower()))

print("\nScholarship Rank List:")
for rank, s in enumerate(eligible, 1):
    print(rank, s[0], "Marks:", s[1], "Attendance:", s[2])

print("\nIneligible Students:", ineligible)
```

5. Sample Input

```
Enter number of applicants: 4

Enter details for student 1
Name: Aarav
Aggregate marks (%): 90
Number of subjects: 2
Subject 1 marks (%): 85
Subject 2 marks (%): 95
Attendance percentage: 90
Annual family income (Rs.): 200000

Enter details for student 2
Name: Bhavya
Aggregate marks (%): 90
Number of subjects: 2
Subject 1 marks (%): 88
Subject 2 marks (%): 92
Attendance percentage: 85
Annual family income (Rs.): 180000

Enter details for student 3
```

```
Name: Charan
Aggregate marks (%): 80
Number of subjects: 2
Subject 1 marks (%): 55
Subject 2 marks (%): 85
Attendance percentage: 90
Annual family income (Rs.): 150000

Enter details for student 4
Name: Divya
Aggregate marks (%): 78
Number of subjects: 2
Subject 1 marks (%): 70
Subject 2 marks (%): 80
Attendance percentage: 80
Annual family income (Rs.): 300000
```

6. Sample Output

```
===== SCHOLARSHIP RANK LIST =====
1. Aarav | Aggregate: 90.00% | Attendance: 90.00% | Income: Rs. 200000.00
2. Bhavya | Aggregate: 90.00% | Attendance: 85.00% | Income: Rs. 180000.00

===== INELIGIBLE STUDENTS =====
Charan: Subject marks below 60%
Divya: Annual income exceeds Rs. 250000
```

Test Case 2

Progressive Electricity Billing System Using Python

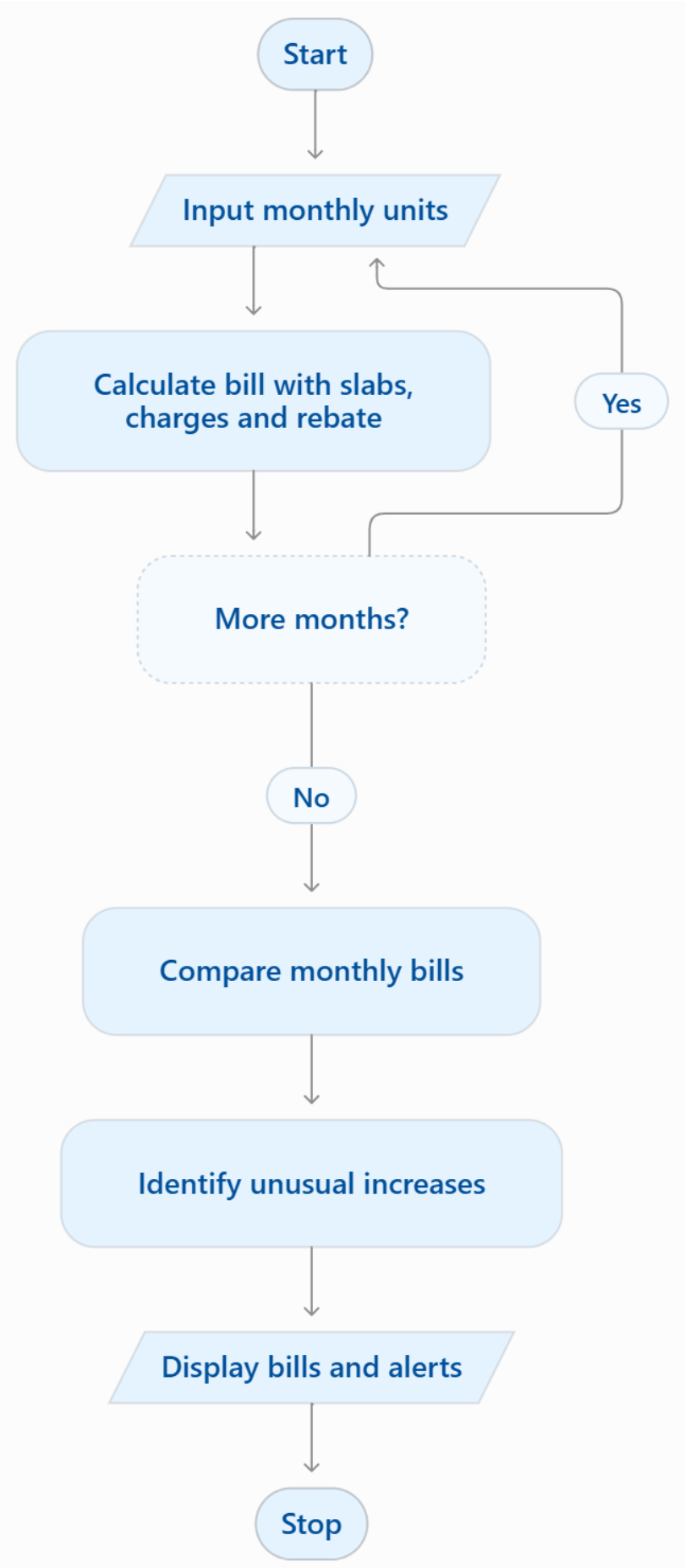
1. Problem Statement

Progressive Electricity Billing: Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

2. Algorithm

1. **Start the program.**
2. **Input the number of months and each month's electricity consumption.**
3. **Calculate energy charges using progressive slabs.**
4. **Add fixed charges and a 5% surcharge.**
5. **Apply the rebate when consumption is at most 100 units.**
6. **Calculate the final bill for each month.**
7. **Compare each month's bill with the previous month's bill.**
8. **If the increase exceeds 20%, mark it as unusual.**
9. **Display monthly consumption, bills, and unusual increases.**
10. **Stop.**

3. Flowchart


Start
Input monthly units
Calculate bill with slabs, charges and rebate
Yes
More months?
No
Compare monthly bills
Identify unusual increases
Display bills and alerts
Stop

4.Source Code

```
n = int(input("Enter number of months: "))
bills = []

for i in range(n):
    units = float(input(f"Units for month {i+1}: "))

    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 200 + (units - 100) * 3
    else:
        energy = 500 + (units - 200) * 5

    surcharge = energy * 0.05
    rebate = energy * 0.10 if units <= 100 else 0
    bill = energy + 50 + surcharge - rebate
    bills.append(bill)

    print(f"Month {i+1} Bill: Rs.{bill:.2f}")

print("\nBill Comparison:")
for i in range(1, n):
    increase = (bills[i] - bills[i-1]) / bills[i-1] * 100

    if increase > 20:
        print(f"Month {i+1}: Unusual increase ({increase:.2f}%)")
    else:
        print(f"Month {i+1}: Normal change ({increase:.2f}%)")
```

5. Sample Input

```
Plain text

Enter number of months: 3
Units for month 1: 80
Units for month 2: 150
Units for month 3: 250
```

6. Sample Output

```
Plain text

Month 1 Bill: Rs.202.00
Month 2 Bill: Rs.417.50
Month 3 Bill: Rs.837.50

Bill Comparison:
Month 2: Unusual increase (106.68%)
Month 3: Unusual increase (100.60%)
```

Taste Case 3

Hospital Billing and Patient Classification Using Python

1. Problem Statement

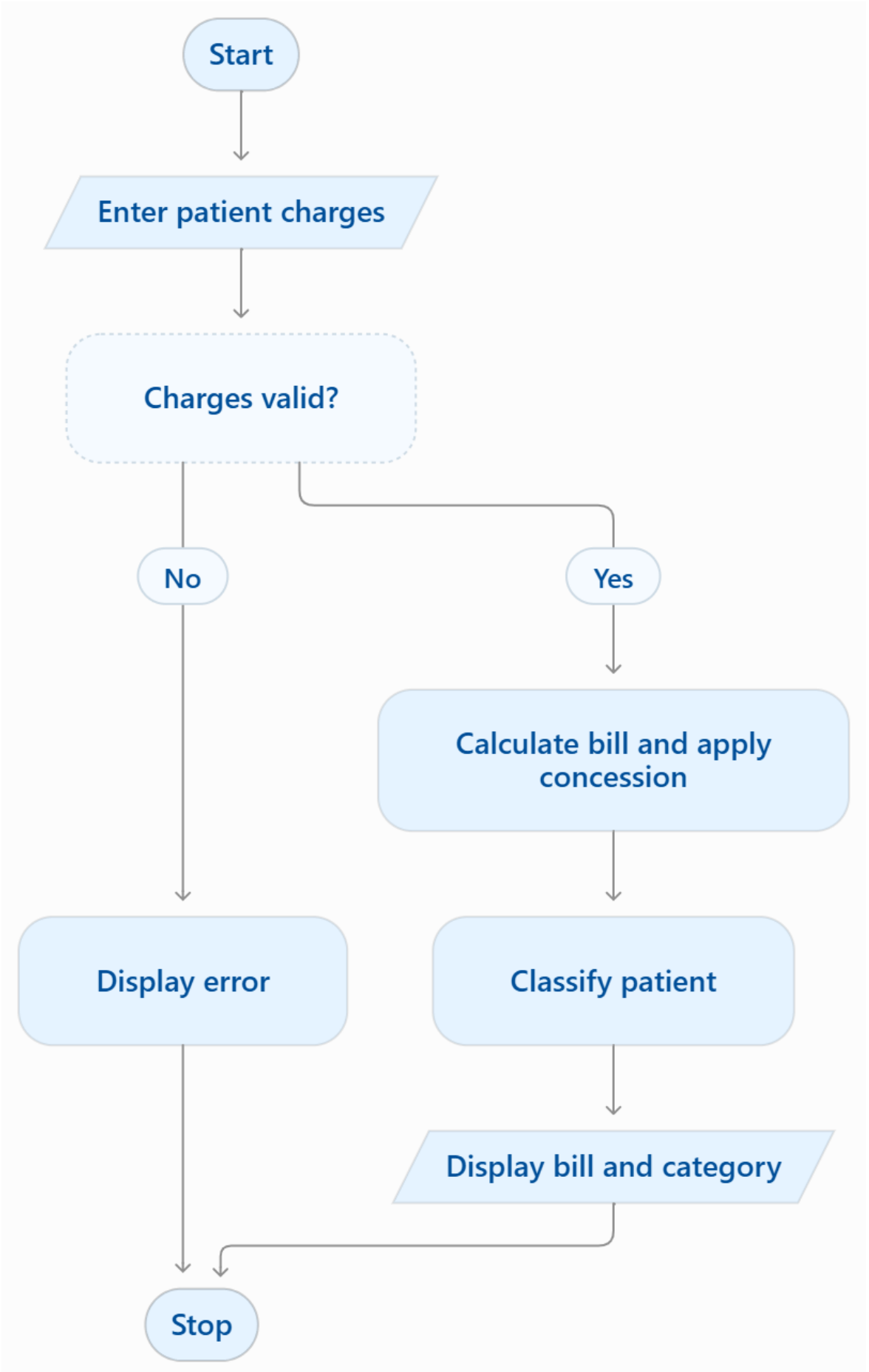
Hospital Billing and Patient Classification: Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

2. Algorithm

1. **Start.**
2. **Input patient name and consultation, diagnostic, and treatment charges.**
3. **Validate that all charges are non-negative.**
4. **Calculate the total bill.**
5. **If the total is at least ₹5,000, apply a 10% concession.**
6. **Calculate the final payable amount.**
7. **Classify the patient based on the final bill.**

8. **Display the total bill, concession, final amount, and classification.**
9. **Stop.**

3. Flowchart


Start
Enter patient charges
Charges valid?
No
Yes
Calculate bill and apply concession
Display error
Classify patient
Display bill and category
Stop

4.Source Code

```
name = input("Patient name: ")
consult = float(input("Consultation charge: "))
diagnostic = float(input("Diagnostic charge: "))
treatment = float(input("Treatment charge: "))

if min(consult, diagnostic, treatment) < 0:
    print("Invalid charges!")
else:
    total = consult + diagnostic + treatment
    concession = total * 0.10 if total >= 5000 else 0
    final_bill = total - concession

    if final_bill < 2000:
        category = "Low Bill"
    elif final_bill <= 10000:
        category = "Medium Bill"
    else:
        category = "High Bill"

    print("\n--- Hospital Bill ---")
    print("Patient:", name)
    print("Total Bill: Rs.", total)
    print("Concession: Rs.", concession)
    print("Final Bill: Rs.", final_bill)
    print("Classification:", category)
```

5. Sample Input

```
Patient name: Ravi
Consultation charge: 1000
Diagnostic charge: 2500
Treatment charge: 3000
```

6. Sample Output

```
--- Hospital Bill ---
Patient: Ravi
Total Bill: Rs. 6500.0
Concession: Rs. 650.0
Final Bill: Rs. 5850.0
Classification: Medium Bill
```

Test Case 4

Railway Fare and Reservation Calculator Using Python

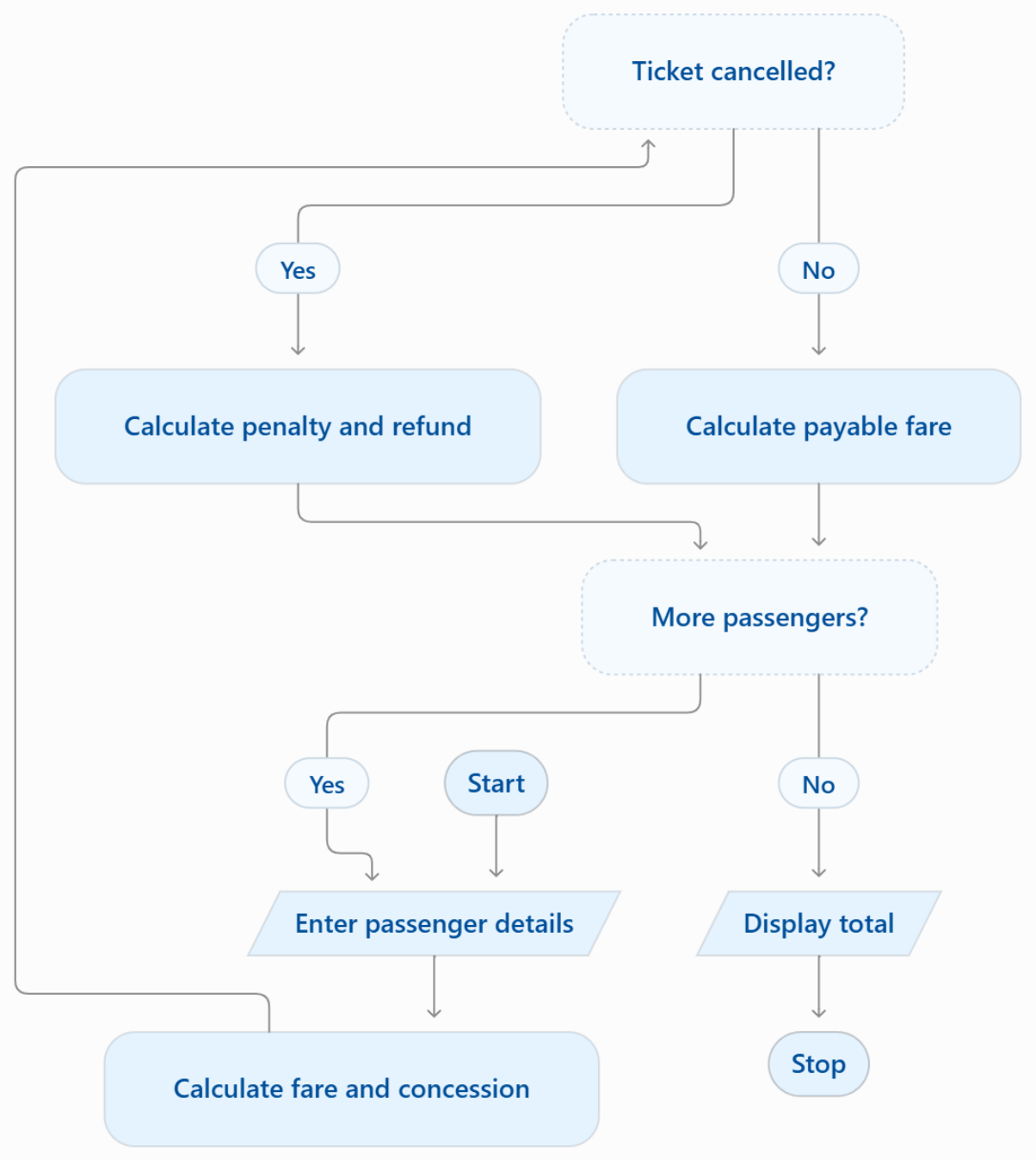
1. Problem Statement

Railway Fare and Reservation Calculator: Calculate fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and specified cancellation penalties.

2. Algorithm

1. Start the program.
2. Input the number of passengers.
3. For each passenger, enter name, age, distance, travel class, and cancellation status.
4. Calculate the base fare using distance slabs.
5. Apply the travel class multiplier.
6. Apply the age concession if eligible.
7. Add the reservation fee.
8. If cancelled, deduct the specified cancellation penalty.
9. Display each passenger's fare and the total amount for all passengers.
10. Stop.

3. Flowchart



4.Source Code

```
n = int(input("Number of passengers: "))
total = 0

for i in range(n):
    age = int(input("Age: "))
```

```
    d = float(input("Distance (km): "))
    cls = input("Class (Sleeper/AC): ").lower()
    cancel = input("Cancel? (yes/no): ").lower()

    fare = (d * 1.5 if d <= 100 else
            150 + (d - 100) * 1.2 if d <= 500 else
            630 + (d - 500))

    if cls == "ac":
        fare *= 2
    if age >= 60:
        fare *= 0.5

    fare += 40

    if cancel == "yes":
        penalty = (fare - 40) * 0.25
        print("Penalty:", round(penalty, 2),
              "Refund:", round(fare - penalty, 2))
    else:
        print("Ticket fare:", round(fare, 2))
        total += fare

print("Total active fare:", round(total, 2))
```

5. Sample Input

```
Number of passengers: 2

Passenger 1
Name: Ravi
Age: 30
Distance (km): 200
Class (Sleeper/AC): Sleeper
Cancel ticket? (yes/no): no

Passenger 2
Name: Sita
Age: 65
Distance (km): 200
Class (Sleeper/AC): AC
Cancel ticket? (yes/no): yes
```

6. Sample Output

```
Passenger 1
Ticket fare: Rs. 310.0

Passenger 2
Cancellation penalty: Rs. 270.0
Refund amount: Rs. 850.0

Total fare for active bookings: Rs. 310.0
```

Test Case 5

Manufacturing Cost and Profitability Analysis Using Python

1. Problem Statement

Manufacturing Cost and Profitability Analysis: Calculate material costs, labour charges, wastage, machine costs, and selling prices; identify whether production meets a specified profit target.

2. Algorithm

1. **Start.**

2. **Input material cost, labour cost, machine cost, wastage percentage, quantity produced, selling price per unit, and profit target.**
3. **Calculate wastage cost.**
4. **Calculate total production cost.**
5. **Calculate total selling revenue.**
6. **Calculate profit.**
7. **Compare profit with the target.**
8. **Display total cost, revenue, profit, and target status.**
9. **Stop.**

3. Flowchart


Start
Input cost and sales details
Calculate total cost and profit
Profit meets target?
Yes
No
Target Achieved
Target Not Achieved
Display results
Stop

4.Source Code

```
material = float(input("Material cost: "))
labour = float(input("Labour cost: "))
machine = float(input("Machine cost: "))
waste = float(input("Wastage percentage: "))
qty = int(input("Quantity produced: "))
price = float(input("Selling price per unit: "))
target = float(input("Profit target: "))

wastage = material * waste / 100
cost = material + labour + machine + wastage
revenue = qty * price
profit = revenue - cost

print("Total Cost:", cost)
print("Revenue:", revenue)
print("Profit:", profit)
print("Status:", "Target Achieved" if profit >= target
    else "Target Not Achieved")
```

5. Sample Input

```
Material cost: 10000
Labour cost: 5000
Machine cost: 3000
Wastage percentage: 10
Quantity produced: 100
Selling price per unit: 250
Profit target: 5000
```

6. Sample Output

```
Total Cost: 19000.0
Revenue: 25000
Profit: 6000.0
Status: Target Achieved
```

Test Case 6

Examination Result and Grace-Mark Validation Using Python

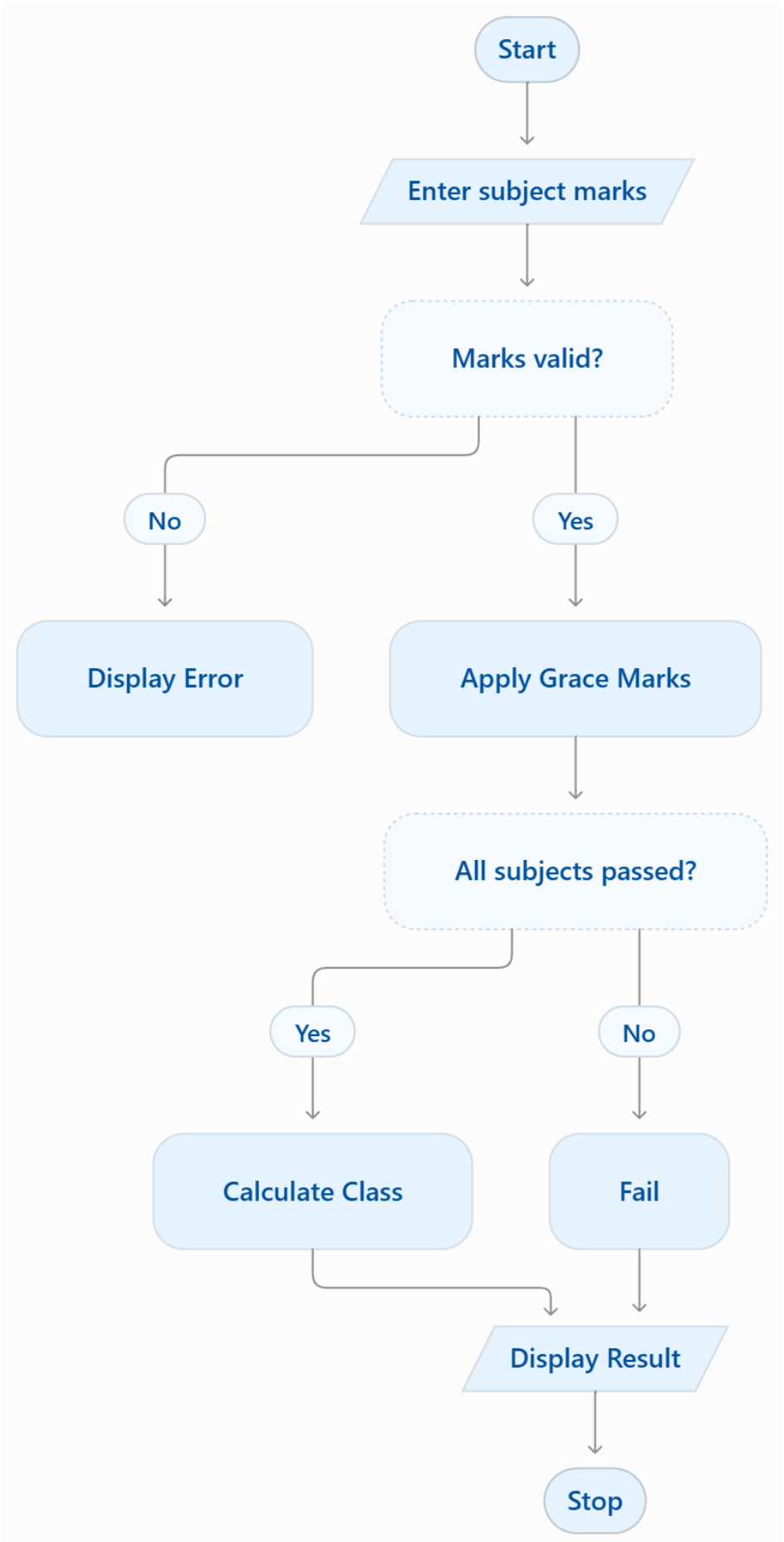
1. Problem Statement

Examination Result and Grace-Mark Validation: Process marks across multiple subjects, apply subject-wise pass criteria and a specified grace-mark policy, and generate final classifications without exceeding maximum marks.

2. Algorithm

1. **Start.**
2. **Input the number of subjects and marks obtained in each subject.**
3. **Validate that each mark is between 0 and 100.**
4. **Check whether each subject meets the passing mark of 35.**
5. **If a subject is short by at most 5 marks, apply the required grace marks.**
6. **Ensure the final marks do not exceed 100.**
7. **Check whether all subjects are passed.**
8. **Calculate the overall percentage and determine the classification.**
9. **Display final subject marks, percentage, and result.**
10. **Stop.**

3. Flowchart


Start
Enter subject marks
Marks valid?
No
Yes
Display Error
Apply Grace Marks
All subjects passed?
Yes
No
Calculate Class
Fail
Display Result
Stop

4.Source Code

```
n = int(input("Number of subjects: "))
marks = list(map(int, input("Enter marks: ").split()))

if len(marks) != n or any(m < 0 or m > 100 for m in marks):
    print("Invalid marks")
else:
    marks = [min(100, m + (35-m if 0 < 35-m <= 5 else 0))
             for m in marks]
    p = sum(marks) / n

    if min(marks) < 35:
        result = "Fail"
    elif p >= 75:
        result = "Distinction"
    elif p >= 60:
        result = "First Class"
    elif p >= 50:
        result = "Second Class"
    else:
        result = "Pass"

    print("Final Marks:", marks)
    print("Percentage:", round(p, 2), "%")
    print("Result:", result)
```

5. Sample Input

```
Number of subjects: 4
Marks in subject 1: 80
Marks in subject 2: 32
Marks in subject 3: 65
Marks in subject 4: 70
```

6. Sample Output

```
Final Marks: [80, 35, 65, 70]
Percentage: 62.5 %
Result: First Class
```

Test Case 7

Campus Event Financial Planning Using Python

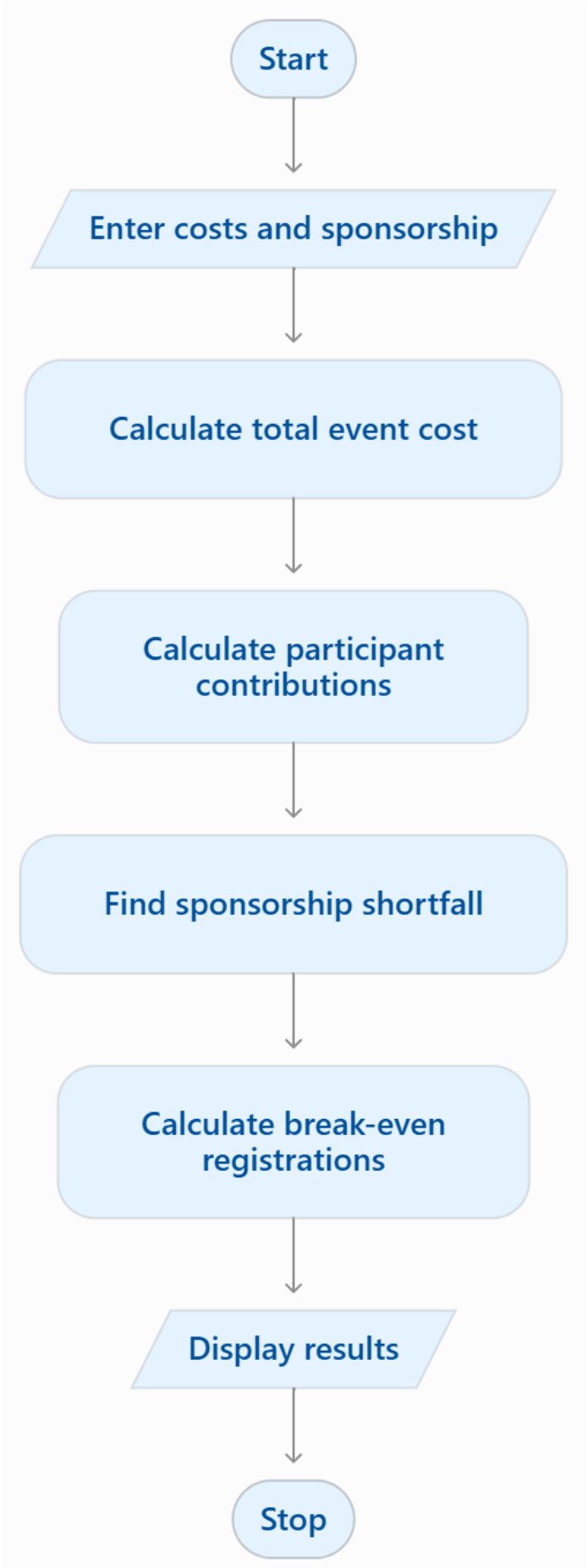
1. Problem Statement

The Campus Event Financial Planning System calculates the total cost of organizing a campus event, including venue, equipment, catering, transport, and certificates. It determines the participant contribution, calculates any sponsorship shortfall, and finds the minimum number of registrations required to break even.

2. Algorithm

1. **Start.**
2. **Input all event expenses and available sponsorship.**
3. **Calculate total event cost.**
4. **Input the contribution per participant.**
5. **Calculate the sponsorship shortfall after expected registrations.**
6. **Calculate the minimum registrations required to break even.**
7. **Display total cost, participant contributions, sponsorship shortfall, and break-even count.**
8. **Stop.**

3. Flowchart


Start
Enter costs and sponsorship
Calculate total event cost
Calculate participant contributions
Find sponsorship shortfall
Calculate break-even registrations
Display results
Stop

4.Source Code

```
import math

venue = float(input("Venue cost: "))
equipment = float(input("Equipment cost: "))
catering = float(input("Catering cost: "))
transport = float(input("Transport cost: "))
certificates = float(input("Certificate cost: "))
sponsor = float(input("Sponsorship amount: "))
participants = int(input("Expected participants: "))
contribution = float(input("Contribution per participant: "))

cost = venue + equipment + catering + transport + certificates
collected = participants * contribution
shortfall = max(0, cost - sponsor - collected)

if contribution > 0:
    breakeven = math.ceil(max(0, cost - sponsor) / contribution)
else:
    breakeven = 0 if sponsor >= cost else "Not possible"

print("\nTotal Event Cost:", cost)
print("Participant Contributions:", collected)
print("Sponsorship Shortfall:", shortfall)
print("Break-even Registrations:", breakeven)
```

5. Sample Input

```
Venue cost: 10000
Equipment cost: 5000
Catering cost: 15000
Transport cost: 4000
Certificate cost: 1000
Sponsorship amount: 10000
Expected participants: 40
Contribution per participant: 500
```

6. Sample Output

```
Total Event Cost: 35000.0
Participant Contributions: 20000.0
Sponsorship Shortfall: 5000.0
Break-even Registrations: 50
```

Test Case 8

ATM Transaction Sequence Simulation Using Python

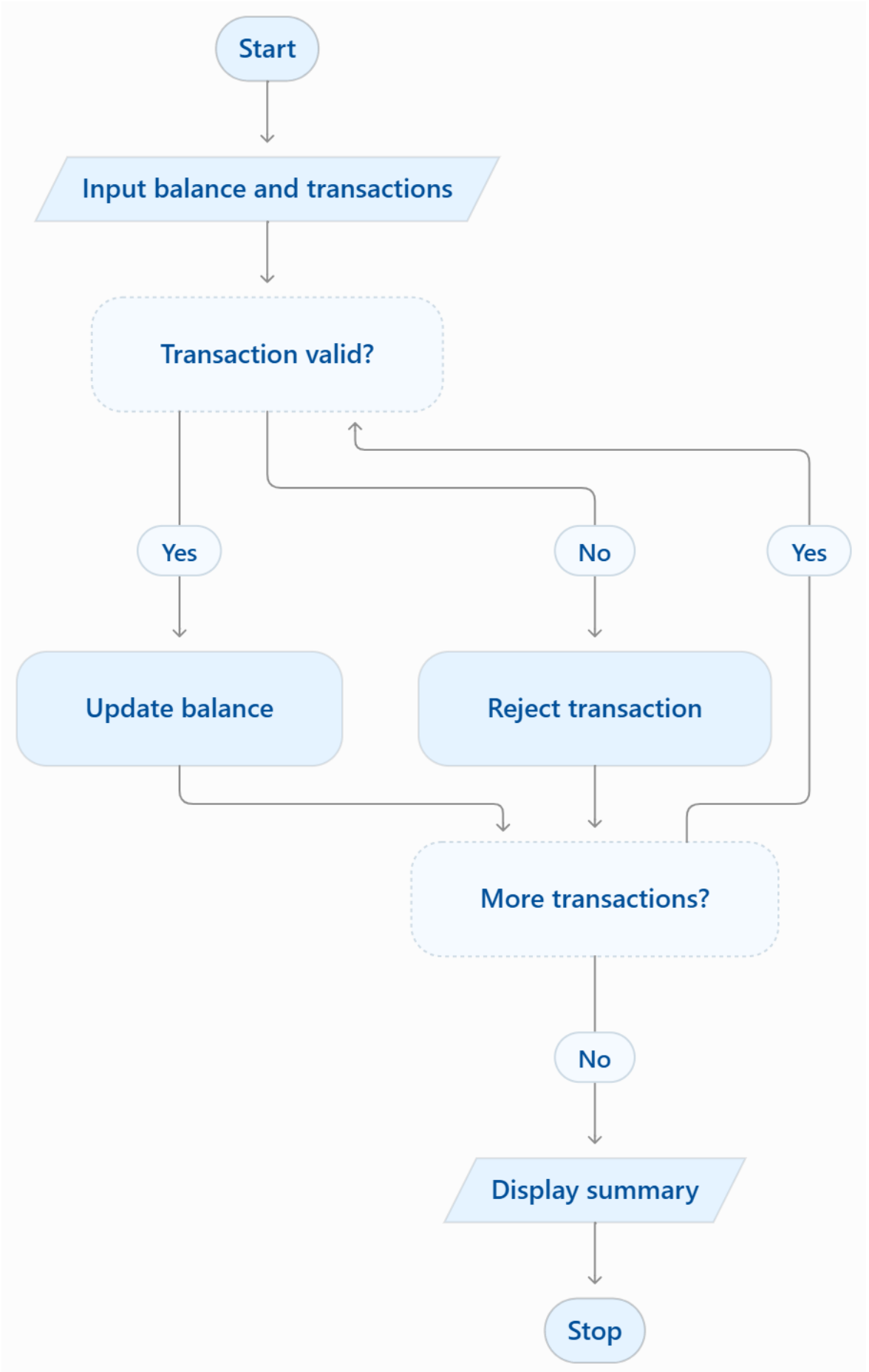
1. Problem Statement

The ATM Transaction Sequence Simulation processes deposits and withdrawals while maintaining a simulated account balance. It enforces a maximum withdrawal limit, rejects invalid transactions, prevents withdrawals exceeding the available balance, and displays a summary of successful and rejected transactions.

2. Algorithm

1. Start.
2. Input the initial account balance and number of transactions.
3. For each transaction, input the type and amount.
4. If the transaction is a valid deposit, add the amount to the balance.
5. If it is a valid withdrawal within the limit and available balance, subtract the amount.
6. Reject invalid transactions.
7. Count successful and rejected transactions.
8. Display the final balance and transaction summary.
9. Stop.

3. Flowchart


Start
Input balance and transactions
Transaction valid?
Yes
No
Yes
Update balance
Reject transaction
More transactions?
No
Display summary
Stop

4.Source Code

```
b = float(input("Initial balance: "))
n = int(input("Number of transactions: "))
s = r = 0

for _ in range(n):
    t = input("Type (deposit/withdraw): ").lower()
    a = float(input("Amount: "))

    if a <= 0 or t not in ("deposit", "withdraw"):
        r += 1
    elif t == "deposit":
        b += a
        s += 1
    elif a <= b and a <= 10000:
        b -= a
        s += 1
    else:
        print("Rejected")
        r += 1

print("Balance:", b, "\nSuccessful:", s, "\nRejected:", r)
```

5. Sample Input

```
Initial balance: 5000
Number of transactions: 3
Transaction (deposit/withdraw): deposit
Amount: 2000
Transaction (deposit/withdraw): withdraw
Amount: 3000
Transaction (deposit/withdraw): withdraw
Amount: 12000
```

6. Sample Output

```
Transaction rejected

Final Balance: 4000.0
Successful Transactions: 2
Rejected Transactions: 1
```

Test Case 9

Smart Parking Fee Management Using Python

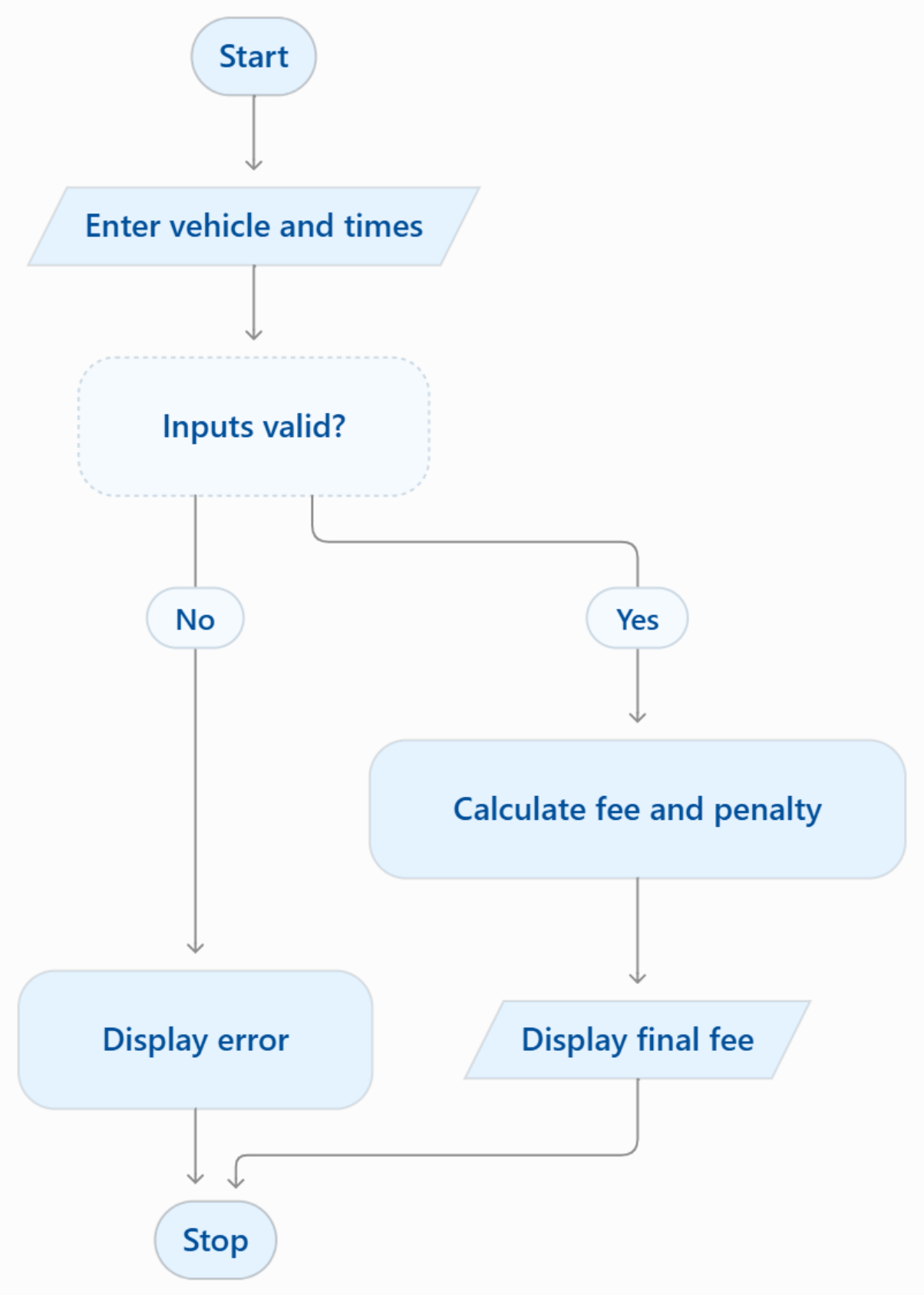
1. Problem Statement

The Smart Parking Fee Management System calculates parking charges based on vehicle category, entry time, exit time, overnight parking, and daily fee limits. It validates time inputs and applies a penalty when the vehicle remains parked for more than 24 hours.

2. Algorithm

1. Start.
2. Input vehicle category, entry time, and exit time.
3. Validate the time format and vehicle category.
4. Calculate the parking duration.
5. Calculate the fee based on vehicle category and hours parked.
6. Apply the daily limit, overnight charge, and penalty if applicable.
7. Display parking duration and final fee.
8. Stop.

3. Flowchart



4.Source Code

```
from datetime import datetime, timedelta
from math import ceil

v = input("Vehicle (bike/car): ").lower()
a = input("Entry time (HH:MM): ")
b = input("Exit time (HH:MM): ")

try:
    s = datetime.strptime(a, "%H:%M")
    e = datetime.strptime(b, "%H:%M")
    if v not in ("bike", "car"):
        raise ValueError("Invalid vehicle")
    if e <= s:
        e += timedelta(days=1)

    h = ceil((e-s).seconds / 3600) if e.date() == s.date() else ceil((e-s).total_seconds()/3600)
    fee = min(h * (20 if v == "bike" else 40), ceil(h/24)*300)
    overnight = s.hour >= 22 or s.hour < 6 or e.hour >= 22 or e.hour < 6
    fee += 50 if overnight else 0
    fee += 200 if h > 24 else 0

    print("Hours:", h, "Final fee: Rs.", fee)
except ValueError:
    print("Invalid input!")
```

5. Sample Input

```
Vehicle (bike/car): car
Entry time (HH:MM): 21:00
Exit time (HH:MM): 23:30
```

6. Sample Output

```
Parking hours: 3
Overnight charge: 50
Final parking fee: Rs. 170
```

Test Case 10

Student Timetable Conflict Detection

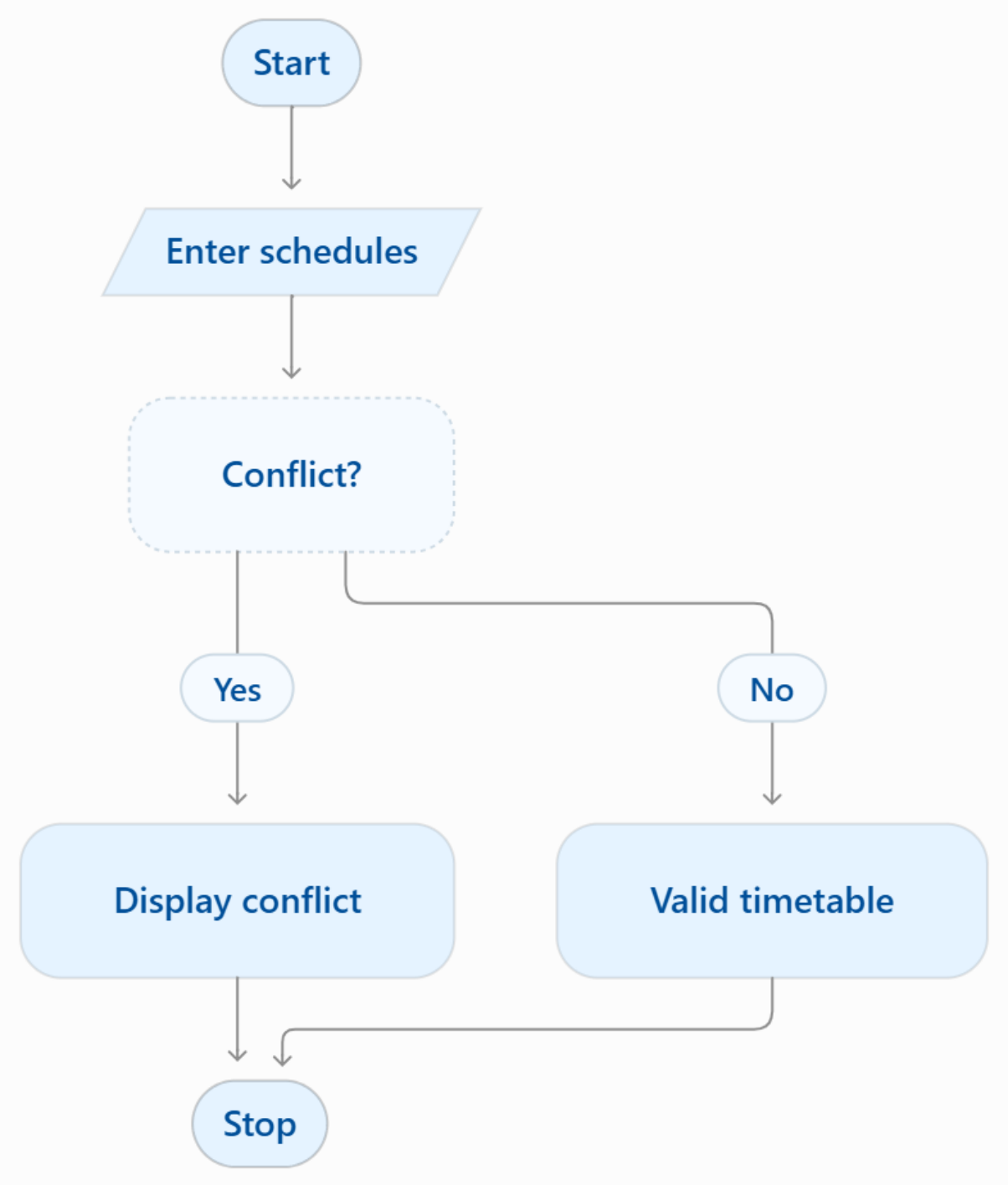
1. Problem Statement

Student Timetable Conflict Detection: Compare course schedules represented by strings and tuples, detect overlapping periods, identify unavailable rooms, and validate a proposed timetable.

2. Algorithm

1. Start the program.
2. Enter course details, time slots, and room numbers.
3. Store each course schedule as a tuple.
4. Compare the time slots of each pair of courses.
5. Check whether the same room is assigned to overlapping classes.
6. Display timetable conflicts or confirm that the timetable is valid.
7. Stop the program.

3. Flowchart



4.Source Code

```
courses = [
   ("Python", 9, 10, "R1"),
   ("Java", 9, 11, "R2"),
   ("Maths", 10, 11, "R1")
]

for i in range(len(courses)):
   for j in range(i + 1, len(courses)):
      a, b = courses[i], courses[j]

      if a[2] > b[1] and b[2] > a[1]:
         print(a[0], "and", b[0], "have a time conflict")

         if a[3] == b[3]:
            print("Room conflict!")
```

5. Sample Input

```
Python  09:00-10:00  R1
Java    09:30-10:30  R2
Maths   10:00-11:00  R1
```

6. Sample Output

```
Python and Java : Time conflict
Java and Maths : Time conflict
```